

Offline Legal Translation — Operating Manual

Version 1.2 · 2026-08-01 · EN↔SL (DE later) · hpelitebook8g1i16, Ubuntu 26.04

This document consolidates and replaces the Hardware Assessment, Workflow Architecture, and Addenda 1 and 2. Version 1.2 revises it for the multi-project confidential layout, the encrypted container, and the Claude Code project location. Everything currently believed correct is here.

1. What this system is

A local pipeline that produces draft translations of legal documents on one machine, with no network egress, so that a certifying translator edits rather than translates from scratch. It is judged on one number only: whether editing a machine draft takes less time than translating the same page fresh.

Everything is arranged around that measurement. Section 6 is the experiment that decides whether to proceed; nothing beyond it should be built until that number is known.

1.1 The four commands you will use most

Command	Purpose
tr-project	List, create, or switch the active confidential project
tr-status	What is translated, what is missing, what is stale
tr-run	Translate everything in source/ not yet in translated/
tr-lint	Deterministic checks; produces the reviewer worklist
tr-xlsx --survey	Measure spreadsheet redundancy before committing to it

Command blocks are colour-coded: **command** *--option* \$VARIABLE "string" # comment

2. Layout and naming

Confidential work lives in one encrypted container holding many projects. Each project is self-contained: its own sources, translations, working files, logs, and translation memory. Filenames are preserved exactly from source to output, so finding gaps is a plain set difference between two directories.

```
~/Claude_Stuff/cli_projects/
translation-tools/      the kit. Outside the data tree.
  bin/ lib/ glossary/ prompts/
~/.translate-venv/      python env. Survives unmount.

~/translation-work/
confidential-projects/  <- ENCRYPTED CONTAINER MOUNTS HERE
  .active               which project the tools operate on
  _shared/
    glossary/base.tsv    terminology reusable across matters
    glossary/nontranslatable.txt
```

prompts/translate.txt	
kranj-2024/	a project
source/	originals. Read-only.
translated/	deliverables. SAME filenames.
work/ocr/	OCR output and text layers
work/tm.sqlite	memory, cache, and resume state
logs/	
glossary/project.tsv	case-specific terms; override base
project.conf	language pair, suffix, OCR languages
maribor-2025/	another project, fully isolated

What is shared and what is not. Glossaries layer: the shared base supplies settled legal terminology, and each project overlays case-specific renderings that win on conflict. Translation memory does NOT layer. Each project keeps its own, because memory holds actual source and target sentences, and those must not cross between matters with different clients and different confidentiality obligations.

Subdirectories under a project's `source/` are mirrored into `translated/`. If a client permits the suffix variant, set it in that project's `project.conf` and every tool follows:

```
TR_SUFFIX=_translated      # in <project>/project.conf
```

2.1 The \$KIT shorthand

The kit path is long, so this manual uses `$KIT` throughout. Define it once and the command blocks below can be copied verbatim.

```
echo 'export KIT=~/.Claude_Stuff/cli_projects/translation-tools' >> ~/.bashrc
source ~/.bashrc
```

The kit deliberately sits on a different branch of the filesystem from the case data. The Claude Code guard refuses to start a session from anywhere inside or above `~/translation-work/`, so the scripts must live outside that tree for `cd $KIT && claude` to be a safe operation. Keeping them under `Claude_Stuff/cli_projects/` also puts all Claude Code work in one place.

Nothing in the kit hardcodes its own location — every script derives it at run time — so the directory can be renamed or moved without edits.

2.2 Where these documents live

Keep the reference documents at `~/translation-work/docs/`, one level above the container — not inside `confidential-projects/`.

The reason is that `confidential-projects/` is a mount point. When the container is closed that directory is empty by design, so documents stored there vanish exactly when they are most needed: at the moment you are trying to remember how to open the container. Documents kept one level up remain available whether the container is open or closed, survive recreating it, and contain no case material, so nothing is lost by holding them outside the encrypted volume.

```
mkdir -p ~/translation-work/docs
# then save the manual, handover, and container documents there
```

The Claude Code deny rules are scoped to confidential-projects/ rather than all of ~/translation-work/, so a session can still read these documents while remaining unable to read any case data.

2.3 Selecting a project

Every tool operates on the active project. This is the one new way the layout can go wrong, so the tools are deliberately noisy about it: tr-run prints a banner and asks for confirmation before touching anything, and any tool refuses outright when no project is set.

```
tr-project          # show active, list all with file counts
tr-project kranj-2024      # switch
tr-project --new maribor-2025 # scaffold and switch
tr-project --none         # clear

# override for a one-off, without changing the active project
TR_ROOT=~/translation-work/confidential-projects/kranj-2024 tr-status
```

The active project is recorded in a file inside the container, not in your shell, so it persists across terminals and is unset automatically whenever the container is closed.

3. One-time setup

Run these once, in order, from your home directory. Total time is dominated by the model download.

3.1 Install the kit

```
# a shorthand worth putting in ~/.bashrc, used throughout this manual
export KIT=~/.Claude_Stuff/cli_projects/translation-tools

mkdir -p "$KIT"
unzip translation-tools.zip -d "$KIT"
chmod +x "$KIT"/bin/*
"$KIT"/bin/tr-setup
source ~/.bashrc           # picks up PATH and TR_PROJECTS
```

tr-setup installs OCR and PDF tooling, builds a Python virtual environment outside the container so it survives unmounting, and warns if swap is too small. It creates no project data — that comes from case-init and tr-project. It is safe to re-run.

Then the container and the guard, both covered in full by the Encrypted Case Container document:

```
cd "$KIT"
./bin/case-init 40G           # LUKS2 container, once

cp bin/case-guard ~/.local/sbin/claude # refuses to start while mounted
chmod +x ~/.local/sbin/claude

case-open
tr-project --new kranj-2024
```

3.2 Increase swap

Do this before any long run. It is a safety net against the out-of-memory killer terminating a multi-day job, not a performance measure; inference should never actually touch swap.

```
sudo swapoff /swapfile
sudo fallocation -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile && sudo swapon /swapfile
free -h           # expect 8.0Gi swap
```

3.3 Install and register the model

GaMS3-12B-Instruct is a Slovene-adapted model from the University of Ljubljana, built on Gemma 3. It is not in the Ollama library, so it comes from Hugging Face as a GGUF. Q8_0 is chosen because quantization damage falls hardest on exactly the sparse morphological patterns Slovene depends on, and throughput is not a constraint here.

```
tr-model           # ~13 GB download, once

# equivalent to:
```

```
# ollama pull hf.co/mradermacher/GaMS3-12B-Instruct-GGUF:Q8_0
# ollama create gams3:q8 -f <Modelfile pinning num_ctx and temperature>

ollama list           # confirm gams3:q8 is present
```

The alias matters. Every script refers to gams3:q8, so switching models later is one tr-model invocation rather than an edit in eight places. It also pins num_ctx and temperature so behavior does not drift with an Ollama upgrade.

Verify the tag at pull time. The repository publishes both static and imatrix-weighted quantizations and tag syntax occasionally changes. If tr-model fails, open the model page, copy the exact tag, and pass it as the first argument: tr-model hf.co/...:Q8_0

3.4 Confidentiality hardening

The model makes no network calls, but two exposures remain and both are more likely to cause a breach than the inference stack.

Confirm the disk is encrypted. Case material on a laptop is exposed by theft, not by the network:

```
lsblk -o NAME,FSTYPE,MOUNTPOINT | grep -i crypt
```

If that returns nothing, the installation is unencrypted. Retrofitting is disruptive; an encrypted container holding only ~/translate is the pragmatic alternative.

Confirm no cloud-routed models. Recent Ollama versions can dispatch to hosted models, which would send document text off the machine:

```
ollama list           # no "-cloud" tagged entries
ollama ps             # what is actually loaded right now
```

3.5 Verify the install

```
echo "Obdolženi je bil obtožen po čl. 211 odst. 2 KZ-1." \
| ollama run gams3:q8 --verbose
```

Read two things from the --verbose output: prompt eval rate (prefill) and eval rate (generation). Expect roughly 4 tokens per second generation on this hardware. Prefill will be considerably faster.

4. Before bulk translation

These three inputs determine output quality far more than any model setting. None can be automated, and all should be settled before the corpus is processed.

4.1 The glossary — highest leverage

Two files, layered. The shared base holds legal terminology that carries across matters; the project file holds case-specific renderings and wins on conflict. The certifying translator fixes each once, up front, and every later deviation is reported by tr-lint with its location.

```
P=~/.translation-work/confidential-projects

nano $P/_shared/glossary/base.tsv      # reusable across cases
nano $P/kranj-2024/glossary/project.tsv # this matter only

# obdolženi<TAB>the accused<TAB>criminal procedure
# oškodovanec<TAB>the injured party
# zaslišanje<TAB>examination<TAB>not "hearing"
```

Promote a term to the base file only once it is settled and not case-specific. Names, entities, and matter-specific usages stay in the project file.

Extract candidates from the corpus rather than inventing them. Charges, statutory references, court and agency names, procedural terms, party designations, document types. The already-installed Qwen model is fast and well suited to drafting the candidate list for the translator to rule on.

4.2 Non-translatables

Regex patterns in nontranslatable.txt mark strings reproduced verbatim: case numbers, file references, dates, amounts, statute short forms. Defaults for Slovene court formats are supplied. Every one of these the model would otherwise "helpfully" translate is a guaranteed edit at a known location, so catching them in code is free quality.

```
nano $P/_shared/glossary/nontranslatable.txt

# test a pattern before trusting it (kit dir, no case data needed):
cd "$KIT" && python3 -c "
import sys; sys.path.insert(0,'lib'); import trlib
print(trlib.is_translatable('I K 12345/2024')) # expect False"
```

4.3 Seed the memory with the translator's prior work

The metric is edits this particular translator makes. If any prior EN↔SL legal translations exist, aligning them into the memory means drafts arrive already using their established renderings and register. A draft that matches their habits produces fewer corrections than an objectively similar draft that does not. This is probably the single highest-value input available and costs one conversation plus an alignment pass.

5. Handling each source type

5.1 Word documents

```
cd ~/translation-work/confidential-projects/kranj-2024
tr-docx source/ovadba.docx translated/ovadba.docx --from sl --to en
```

Paragraphs, headings, lists, tables, headers and footers are preserved. Each paragraph is split into sentences before translation, so the memory keys on sentences and repeats are reused across the whole corpus.

Known limitation. Formatting that varies within a single paragraph — one bold word mid-sentence — is not preserved; the paragraph takes the formatting of its first run. Paragraph-level styling is intact. If intra-paragraph emphasis is legally significant in this corpus, use OmegaT for those files instead (Section 9).

5.2 Spreadsheets

Always survey first. Large tables are dominated by repeated categorical values, and deduplication both collapses the work and guarantees identical cells receive identical translations.

```
# measure first, translate nothing
tr-xlsx source/tabela.xlsx --survey

# then translate; each distinct string is translated once
tr-xlsx source/tabela.xlsx translated/tabela.xlsx

# restrict to specific columns if only some hold text
tr-xlsx source/tabela.xlsx translated/tabela.xlsx --cols B,D,E
```

The survey prints the unique-string ratio and an estimate of translation time with and without deduplication. On the 7,000-row table, run this before planning anything around it. Output stays in XLSX, which is correct — ten columns will not transfer usefully into Word.

5.3 Scanned PDFs — two stages, deliberately

This is the highest-risk path in the system, for a specific reason: a model fed corrupted OCR does not flag the corruption. It produces a fluent, confident translation of the corruption. A misread digit in a date or an amount arrives looking entirely correct.

```
# stage 1 - OCR only, then stop
tr-pdf --ocr-only source/zapisnik.pdf

# read the text layer against the page images
less work/ocr/zapisnik.txt
xdg-open work/ocr/zapisnik.ocr.pdf

# stage 2 - only after verification
tr-pdf source/zapisnik.pdf translated/zapisnik.docx
```

Check names, dates, case numbers, amounts, and the characters č, š, ž specifically. Mangled diacritics can still produce valid Slovene words, so no spell-check will catch them. OCR output is cached, so stage 2 does not re-run it.

A second engine is worth adding if scan quality is poor. The installed Qwen model reports a vision capability; a vision model reads a page by a completely different mechanism than Tesseract, so their failure modes are largely uncorrelated. Run both, compare only the numbers, dates, and capitalized tokens, and inspect where they disagree. Test that vision input works before designing around it:

```
ollama run qwen3.6 "Transcribe this page verbatim." ./page.png
```

6. Phase 1 — the experiment that decides everything

Do not build further until this number exists. The comparison is deliberately simple.

Step	Action	Detail
1	Pick six comparable pages	Three for machine assistance, three matched for scratch translation. Same document type, similar density
2	Draft the three	tr-run, or tr-txt --bilingual for side-by-side review
3	Time the translator on both sets	Minutes per page, MT-assisted versus from scratch
4	Compute the ratio	This is the decision. Anything at or above 1.0 means stop
5	Ask which errors cost the time	Terminology? Register? Numbers? That tells you what to fix next

```
case-open && tr-project kranj-2024
cp /path/to/test/*.docx \
~/translation-work/confidential-projects/kranj-2024/source/
tr-status
tr-run
tr-lint

# bilingual layout for side-by-side review (source in grey above target)
tr-txt work/ocr/sample.txt /tmp/sample-bilingual.docx --bilingual
```

Step 5 matters as much as step 4. A ratio slightly above 1.0 caused by one fixable error class — say, untranslated case numbers — is a different verdict than the same ratio caused by systematically wrong register.

7. Daily operation

```
case-open          # unlock the container
tr-project         # confirm which project is active

tr-status          # what remains
tr-status --missing # bare paths, for piping
tr-status -v       # include the completed list

tr-run -n          # dry run: what would be processed
tr-run            # everything missing
tr-run source/one.docx # a single file

tr-lint            # after any run

case-close        # lock up when finished
```

tr-run is resumable at two levels. It skips files already present in translated/, and within a file every segment already in the memory is reused rather than regenerated. Interrupting it costs at most one segment. Re-running after editing the glossary is cheap for the same reason.

A file is re-translated only if its source is newer than its output. To force a redo, delete the output. To force a redo ignoring the memory, change TR_PROMPT_VERSION, which is part of the cache key:

```
rm translated/ovadba.docx && tr-run # reuse memory
TR_PROMPT_VERSION=v2 tr-run # ignore memory, retranslate
```

8. The lint report

tr-lint runs no model, takes seconds, and catches the error classes a language model is structurally worst at noticing. Its output is not a pass/fail gate; it is a prioritized worklist to hand the translator alongside the drafts.

Check	Meaning
FAIL	The segment errored out and contains no translation. Fix and re-run first
NUM	A number in the source is absent from the target, or a number appears that was not in the source. Highest consequence class in legal evidence
NONTR	A string marked non-translatable was altered or dropped
INCON	The same source sentence was translated two different ways. A corpus-level finding no per-document review will surface
GLOSS	An agreed glossary term was not used
ECHO	Target identical to source — possible untranslated passthrough
EMPTY	Empty target

```
tr-lint
tr-lint -tsv work/for-translator.tsv

# open the report in a spreadsheet for the reviewer
libreoffice --calc work/lint-report.tsv
```

Findings are sorted by severity, so the top of the file is where to start. Numeric comparison normalizes separators, so 1.234,56 and 1,234.56 are treated as the same number.

9. When to use OmegaT instead

The scripts above are the fast path: they produce deliverables directly, with filenames preserved, which is what the client wants. They are the right tool for Phase 1 and for the spreadsheet.

OmegaT is the better container for sustained production work, for three reasons: it preserves intra-paragraph formatting properly, it gives the translator a sentence-by-sentence review interface with source and target aligned, and it accumulates a translation memory they own and can reuse on later matters.

The two coexist. Pre-translate with these scripts, export the memory as TMX, and import it into an OmegaT project so segments arrive pre-populated. That path requires no plugin and works with any CAT tool the translator prefers, including Trados.

```
sudo apt install omegat

# export the accumulated memory as TMX for import
cd "$KIT" && python3 -c "
import sys,sqlite3,html; sys.path.insert(0,'lib'); import trlib
db=sqlite3.connect(trlib.path('work','tm.sqlite'))
rows=db.execute('SELECT src,tgt,direction FROM tm').fetchall()
print('<?xml version=\'1.0\' encoding=\'UTF-8\'?>')
print('<tmx version=\'1.4\'><header srclang=\'sl\' '
      'segtype=\'sentence\' datatype=\'plaintext\'><body>')
for s,t,d in rows:
    a,b=d.split('-')
    print(f'<tu><tuv xml:lang=\'{a}\'><seg>{html.escape(s)}</seg></tuv>'
          f'<tuv xml:lang=\'{b}\'><seg>{html.escape(t)}</seg></tuv></tu>'
          print('</body></tmx>')"
```

The TMX export above is a sketch and should be validated against OmegaT before relying on it.

10. Long unattended runs

A job running for days has failure modes a short one does not.

```
# prevent suspend from silently halting the run
systemd-inhibit --what=idle:sleep:handle-lid-switch \
               --why="translation batch" tr-run

# or run detached and reattach later
sudo apt install tmux
tmux new -s tr
tr-run          # then Ctrl-b d to detach
tmux attach -t tr

# monitor from another terminal
watch -n5 "sensors | grep -i 'Package id'; free -h | head -2; ollama ps"
tail -f "$(tr-project | awk '/^active/{print $2}')" /logs/run-*.log
```

Keep the machine on mains power and elevated for airflow. A 15 W mobile processor at sustained full load for days is outside its normal duty cycle. If the firmware exposes a charge limit, cap it so the battery is not held at full charge continuously:

```
cat /sys/class/power_supply/BAT0/charge_control_end_threshold
echo 80 | sudo tee /sys/class/power_supply/BAT0/charge_control_end_threshold
```

11. Troubleshooting

Symptom	Cause and fix
no active project	Run <code>tr-project <name></code> . Tools refuse rather than guess
container not mounted	Run <code>case-open</code> . When closed the path is empty by design
Wrong project translated	<code>tr-run</code> prints a banner and asks first; read it. Delete the wrong outputs and re-run
Connection refused	Ollama is not running. <code>systemctl status ollama</code> ; <code>sudo systemctl start ollama</code>
[TRANSLATION FAILED] in output	Model unreachable or timed out. Fix the cause, then re-run; cached segments are not regenerated
Process killed mid-run	Out of memory. Check swap (S3.2) and lower <code>TR_NUM_CTX</code>
Output contains commentary	Model ignored the output-only instruction. Lower temperature, or tighten prompts/translate.txt
Sentences split at abbreviations	Add the abbreviation to <code>ABBREV</code> in <code>lib/trlib.py</code> and re-run with a new <code>TR_PROMPT_VERSION</code>
Case numbers being translated	Add a pattern to <code>nontranslatable.txt</code> ; verify with <code>is_translatable()</code> per S4.2
Very slow	Confirm mains power and that the platform profile is not power-saver: <code>cat /sys/firmware/acpi/platform_profile</code>
Diacritics wrong in OCR	Confirm <code>-l slv+eng</code> was used. Set <code>TR_OCR_LANGS</code> if the pair differs

12. Environment variables

Variable	Default	Purpose
<code>TR_PROJECTS</code>	<code>~/translation-work/</code> <code>confidential-projects</code>	Container root holding all projects
<code>TR_ROOT</code>	(active project)	Override to target one project for a single command
<code>TR_MODEL</code>	<code>gams3:q8</code>	Model alias used by every script
<code>TR_SRC</code> / <code>TR_TGT</code>	<code>sl</code> / <code>en</code>	Per-project, in <code>project.conf</code> . Use <code>de</code> for German
<code>TR_SUFFIX</code>	(empty)	Per-project, in <code>project.conf</code> . Set if the client requires it
<code>TR_NUM_CTX</code>	<code>8192</code>	Context window. Lower if memory is tight
<code>TR_PROMPT_VERSION</code>	<code>v1</code>	Part of the cache key. Bump to force retranslation
<code>TR_OCR_LANGS</code>	<code>slv+eng</code>	Tesseract languages. Add <code>deu</code> for German

Variable	Default	Purpose
TR_OLLAMA	http://127.0.0.1:11434	Ollama endpoint

For a German matter, set the pair once in that project's project.conf rather than on the command line, so every later run inherits it:

```
tr-project --new berlin-2026
nano ~/translation-work/confidential-projects/berlin-2026/project.conf

TR_SRC=de
TR_TGT=en
TR_OCR_LANGS=deu+eng
```

German is high-resource and well served by base Gemma; GaMS3 continual pre-training on Slovene may have eroded it. Compare against gemma3:12b on a German sample before relying on GaMS3 for that pair.

13. Decisions and why

Decision	Chosen	Reason
Model	GaMS3-12B-Instruct	Slovene-specific; outperforms base Gemma 3 12B on EN→SL and rivals GPT-4o on the Slovene arena
Quantization	Q8_0	Quantization damage is disproportionate for less-resourced languages; throughput is not a constraint
No RAM upgrade	32 GB retained	Workload is bandwidth-bound. More capacity adds no bandwidth and only enables slower, larger models
Segment granularity	Sentence	Matches the translator's review unit and makes the memory reusable across documents
Abbreviation handling	Custom list	Default splitters shatter Slovene legal text at št., čl., odst., which fights the reviewer on every page
Spreadsheet strategy	Unique-string map	Collapses the work and guarantees identical cells translate identically
Verification	Deterministic linter	Catches numeric and consistency errors that models cannot self-detect; costs nothing to run
Back-translation	Dropped	Its value assumed a reviewer reading target-only. Sentence-by-sentence comparison against source catches the same errors directly
Project isolation	Separate memory per matter	Memory holds real sentences. Sharing it across clients would move content between matters
Glossary layering	Shared base + overlay	Terminology is reusable; case specifics are not. Layering gets the benefit without the leak
Data location	Encrypted container	Makes the assistant boundary structural rather than remembered

14. Open items

- Measure the 7,000-row spreadsheet with `tr-xlsx --survey`. The unique ratio determines whether it is hours or days.
- Trial the container on a throwaway 1 GB image and exercise open, close, and the guard before real data goes near it.
- Confirm full-disk encryption as well as the container (S3.4). They cover different threats and both are wanted.
- Test whether Qwen vision input works, for the dual-engine OCR cross-check (S5.3).
- Seed the memory from the translator's prior work if any exists (S4.3).
- Verify German quality against base Gemma before extending to that pair (S12).